

CSCE 608 DATABASE SYSTEMS · MODULE 13 · LECTURE NOTES

Distributed Databases

What you gain, what you give up, and whether you need it.

MODULE OUTCOMES

- State CAP precisely and explain what it does not say.
- Explain PACELC and why it is the more useful framing.
- Explain two-phase commit and its blocking failure.
- Explain consensus and what Raft provides.
- Judge whether a workload needs distribution at all.

1 Why distribute

Reason	Assessment
Capacity — the data does not fit on one machine.	Legitimate, and much rarer than claimed. See below.
Availability — survive the loss of a machine.	Usually the real reason , and a good one. Note that this needs <i>replication</i> , which is a far smaller step than sharding.
Geography — serve users from nearby.	Legitimate and unavoidable when users are on several continents. The speed of light is not negotiable.
Throughput — more machines serving more queries.	Often a single machine suffices once the obvious inefficiencies are addressed. Worth measuring before assuming.
Resume-driven development	It happens, and it is worth being honest that it happens.

A SINGLE MACHINE IS LARGER THAN MOST PEOPLE ASSUME

A commodity server available today has on the order of 128 cores, 1 to 2 terabytes of RAM, and tens of terabytes of NVMe storage delivering millions of IOPS.

That configuration handles workloads that genuinely required a cluster fifteen years ago — and handles them with full ACID transactions, arbitrary joins, a single process to profile, and a single machine to restart.

What distribution costs: joins across shards become slow or unsupported; transactions across shards require coordination or are unavailable; network partitions become a routine operational event rather than a disaster; and the operational complexity is permanent — it does not diminish once the system is built.

So measure before distributing. 'We might need to scale eventually' is not a measurement, and the cost of distributing unnecessarily is paid every day by everyone who works on the system, while the cost of distributing later is paid once by a team that by then understands the workload.

2 CAP and PACELC

CAP, STATED PRECISELY

In the presence of a network partition, a distributed system must choose between consistency and availability. That is the theorem.

It is not 'pick two of three'. Partitions are not something you choose; they are something that happens to you. The theorem describes the choice you face *when one occurs*, which is a much narrower and more useful claim.

And the terms are specific. 'Consistency' here means **linearizability** — every read returns the most recent write, as though there were a single copy. 'Availability' means *every* request to a non-failed node receives a response. Both are stronger than most real systems need or provide.

So the 'CP' and 'AP' labels convey far less than they appear to, and most marketing use of CAP is either imprecise or wrong. Brewer himself wrote a retrospective saying as much.

PACELC IS THE MORE USEFUL FRAMING

Daniel Abadi's formulation: **if there is a Partition, choose between Availability and Consistency; Else, choose between Latency and Consistency.**

The 'else' clause is the important addition. Partitions are rare — perhaps a few times a year. **The latency/consistency trade is paid on every single request, forever.**

Strong consistency requires coordination: a write must reach a quorum before it is acknowledged. Within a datacentre that is a millisecond or two; across regions it is tens to hundreds of milliseconds, bounded below by the speed of light.

So the everyday cost of consistency is latency, not availability. That is the trade an architect actually makes, and CAP's focus on the rare case obscures it. PACELC's contribution is putting the common case in the name.

3 Distributed transactions

```
PHASE 1 -- PREPARE
  coordinator -> all participants: "can you commit?"
  each participant: make the change DURABLE but do not commit;
                    reply YES (now BOUND) or NO
```

```
PHASE 2 -- COMMIT or ABORT
  all YES -> coordinator broadcasts "commit"
  any NO -> coordinator broadcasts "abort"
```

THE BLOCKING PROBLEM: a participant that voted YES and then loses the coordinator CANNOT decide alone. It cannot commit (someone may have voted NO) and cannot abort (others may already have committed). It holds its locks and waits.

TWO-PHASE COMMIT IS CORRECT AND BLOCKING

The protocol is correct: no execution results in some participants committing while others abort.

But a participant that has voted yes has surrendered its autonomy. It has made its changes durable and promised to commit if asked; it cannot unilaterally abort, because others may have been told to commit, and it cannot unilaterally commit, because someone may have voted no.

So if the coordinator fails after collecting votes and before announcing the outcome, every prepared participant holds its locks and waits — indefinitely, until the coordinator recovers and consults its log. Those locks block other transactions, so the failure of one machine stalls work on all the others.

This is the real limitation, and it is structural rather than an implementation flaw. It is why distributed transactions have the reputation they do.

THREE-PHASE COMMIT DOES NOT SOLVE IT IN PRACTICE

3PC inserts an additional round — 'pre-commit' — so that a participant that loses the coordinator can determine the outcome by consulting its peers.

It assumes a synchronous network with a known bound on message delay, so that the absence of a message within the bound proves failure.

Real networks are asynchronous — a message may be arbitrarily delayed and then arrive — and under asynchrony 3PC is not correct. It can produce inconsistent outcomes when a slow node is mistaken for a failed one. The additional round also costs latency on every transaction, including the overwhelming majority that encounter no failure.

So it is essentially unused. The practical answer is to make the coordinator itself fault-tolerant using consensus (§4), so that it never becomes unavailable in the first place. This is what Spanner, CockroachDB, and their successors do, and it is why consensus and distributed transactions are studied together.

4 Consensus

WHAT CONSENSUS PROVIDES, AND ITS LIMIT

A set of machines agrees on a sequence of values, tolerating crashes and lost or delayed messages, provided a majority remains reachable.

In a database it is used for the replicated log (Module 11): if every replica agrees on the same log in the same order, every replica reaches the same state. The log, once again, turns out to be the right primitive.

Raft was designed explicitly for understandability, after a decade in which Paxos proved notoriously difficult to implement correctly — the original paper leaves substantial gaps between the algorithm and a working system. Raft separates leader election, log replication, and the safety argument into distinct, individually comprehensible mechanisms. That is a legitimate and unusual design goal, and it worked: Raft implementations are widespread.

The FLP impossibility result shows that consensus cannot be guaranteed in a fully asynchronous system with even one faulty process. **Real systems use timeouts** and thereby sacrifice guaranteed *liveness* — progress may stall — while preserving *safety*: they never produce an inconsistent result. Stalling is recoverable; inconsistency is not.

Approach	Examples	Trade
Single leader with replicas	PostgreSQL streaming replication, MySQL.	Simple and well understood. Failover is the hard part — deciding the leader is dead, and avoiding two leaders.
Consensus replication	Spanner, CockroachDB, TiDB, etcd.	Strong consistency with automatic failover , at the cost of a coordination round trip on every write.
Leaderless quorum	Dynamo, Cassandra, Riak.	Highly available and eventually consistent. Conflicts must be resolved by the application or by CRDTs.
Sharding	Vitess, Citus, application-level sharding.	Scales writes by partitioning the key space. Cross-shard queries and transactions are the difficulty , and the shard key choice is effectively permanent.

Spanner's TrueTime deserves a mention: Google bounded clock uncertainty using GPS receivers and atomic clocks in every datacentre, so a transaction can simply wait out the uncertainty interval and thereby achieve externally consistent global timestamps. **It is a software problem solved with hardware**, which few organisations can replicate — though cloud providers now expose bounded-uncertainty clocks, which is beginning to change that.

WHAT YOU GIVE UP, STATED PLAINLY

Joins across shards are slow, restricted, or unsupported. Most systems require that joined tables be co-located by the shard key, which constrains the schema permanently.

Transactions across shards require a coordination round trip at best, and many systems simply do not offer them — pushing the problem to the application, which solves it worse.

Secondary indexes are either local to each shard, requiring a scatter-gather query across every shard to use them, or global, requiring a distributed update on every write. Neither is good.

Debugging becomes distributed tracing. A bug can live in the *interaction* between machines rather than in any one of them, and it may be unreproducible because it depends on timing.

And none of this gets easier later. These are the permanent costs of the decision, paid continuously — which is why §1 argues for measuring before committing to them.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Kleppmann — Designing Data-Intensive Applications	The best single book on this material. Chapters 5 through 9 cover everything in this module with more care and more examples.
Abadi — Consistency Tradeoffs in Modern Distributed Database System Design (PACELC, free)	The §2 framing, in the original. Short.
Ongaro & Ousterhout — In Search of an Understandable Consensus Algorithm (Raft, free)	Raft. The paper is readable, and the site has an animated visualisation that makes leader election obvious.
Corbett et al. — Spanner (free)	TrueTime and externally consistent distributed transactions. Read it for the clock argument.
Jepsen — analyses of real distributed databases	What systems actually do under partition, as opposed to what they claim. Essential reading before trusting any consistency claim.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 For a workload you know, estimate whether a single large machine would suffice. State the numbers: data size, write rate, query rate.

- 2 Benchmark a single PostgreSQL instance on a modern machine and find its actual limits for your workload before assuming it needs distributing.
- 3 State CAP precisely in your own words, then find three online claims about it that are wrong and say why.
- 4 Classify five systems you know using PACELC and justify each classification.
- 5 Implement two-phase commit across three processes. Then kill the coordinator after the prepare phase and observe the participants block.
- 6 Measure how long participants hold locks in that scenario and what else it blocks.
- 7 Implement Raft leader election. Kill the leader and verify a new one is elected within the expected time.
- 8 Measure write latency for a single node, a three-node consensus group in one datacentre, and a simulated cross-region group.
- 9 Set up a two-shard system and write a query requiring a cross-shard join. Implement it and measure the cost against the co-located version.
- 10 Read one Jepsen analysis in full and write a page on what the system claimed, what it actually did, and why the gap existed.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give five reasons people distribute and assess each.
- 2 What does a single modern server handle, and what does distribution cost?
- 3 State CAP precisely and give two things it does not say.
- 4 State PACELC and explain why the else clause matters more.
- 5 Describe two-phase commit and explain the blocking problem exactly.
- 6 Why does three-phase commit not solve it in practice?
- 7 What does consensus provide, and what does Raft contribute over Paxos?
- 8 What does FLP say, and what do real systems sacrifice?
- 9 Compare four replication and partitioning approaches.
- 10 Name four things you permanently give up by distributing.